A

PROJECT REPORT

ON

EaseCollaborate

Submitted By

SHAIK SHAREEF (R200862)
BOGGULA SAI ANUDEEP (R200699)
BOJJA MANJUNATH REDDY (R200510)

Under The Guidance of

S. RAJESWARI

Department of Computer Science and Engineering

Rajiv Gandhi University of Knowledge Technologies
R K Valley, Y.S.R. Kadapa, Andhra Pradesh-516330

DECLARATION

We hereby declare that the project report entitled **"EaseCollaborate"** submitted to the Department of COMPUTER SCIENCE AND ENGINEERING is a genuine work carried out by our team under the guidance of **S.Rajeswari**.This project is submitted in partial fulfillment of the requirements for the award of the **degree of BACHELOR OF TECHNOLOGY in Computer Science and Engineering during the Academic Year 2025-2026**. This project is the result of our own effort and that it has not been submitted to any other University or institution for the award of any degree or diploma other than specified above.

WITH SINCERE REGARDS,

Shaik Shareef (R200862)

Boggula Sai Anudeep (R200699)

Bojja Manjunath Reddy(R200510)

RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES
ANDHRA PRADESH
Accredited by 'NAAC' with 'B+' Grade
(Established through Act No.18 of 2008 by the Govt. of Andhra Pradesh and recognized as per Section 2(f), 12(B) of UGC Act, 1956)
RGUKT RK Valley
Vempalli (M), YSR District – 516330

CERTIFICATE

This is to certify that the project work titled **"EaseCollaborate"** is a bonafied project work submitted by **SHAIK SHAREEF (R200862), BOGGULA SAI ANUDEEP (R200699), BOJJA MANJUNATH REDDY (R200510)** in the department of COMPUTER SCIENCE AND ENGINEERING in partial fulfilment of requirements for the award of degree of Bachelor of Technology in **Computer Science and Engineering** for the year 2025-2026 carried out the work under the supervision.

Signature of Internal Guide
S. RAJESWARI
RGUKT, RK VALLEY

Signature of HOD
Dr. CH. RATNA KUMARI
Assistant Professor
RGUKT, RK VALLEY

Signature of External Examiner

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion ofany task would be incomplete without the mention of the people who made it possible and whose constant guidance and encouragement crown all the efforts success.

I am extremely grateful to our respected Director, **Dr. A V S S KUMARA SWAMI GUPTA** for fostering an excellent academic climate in our institution.

I also express my sincere gratitude to our respected Head of the Department **Dr. CH. RATNA KUMARI** for her encouragement, overall guidance in viewing this project a good asset and effort in bringing out this project.
I would like to convey thanks to our guide at college **S. RAJESWARI** for her guidance, encouragement, co-operation and kindness during the entire duration of the course and academics.

My sincere thanks to all the members who helped me directly and indirectly in the completion of project work. I express my profound gratitude to all our friends and family members for their encouragement.

With our Sincere Regards

Shaik Shareef (R200862)

Boggula Sai Anudeep (R200699)

Bojja Manjunath Reddy (R200510)

CONTENTS

Table Of Contents :

ABSTRACT

EaseCollaborate is a full-stack web-based project management and collaboration platform designed to streamline task organization, improve team coordination, and enhance productivity through intelligent insights. In modern collaborative environments, managing tasks, tracking progress, and ensuring accountability across multiple team members can be challenging when using fragmented tools. EaseCollaborate addresses this problem by providing a centralized system where users can create workspaces, manage projects, assign tasks, and monitor progress in a structured and efficient manner.

The system is developed using a modern technology stack, with a React and TypeScript-based frontend ensuring a responsive and interactive user interface, and a Node.js and Express-based backend providing a scalable RESTful API. MongoDB is used as the database to efficiently store and manage hierarchical data such as users, workspaces, projects, and tasks. The platform incorporates secure authentication using JSON Web Tokens (JWT), along with essential account management features such as email verification and password recovery.

EaseCollaborate supports role-based access control, enabling different levels of permissions at both workspace and project levels. This ensures that users can only perform actions appropriate to their roles, thereby maintaining system integrity and security. The application also includes advanced task management capabilities such as task assignment, priority setting, subtasks, comments, watchers, and activity tracking.

Overall, EaseCollaborate demonstrates the practical implementation of modern full-stack development principles, combining frontend and backend technologies with database design, authentication mechanisms, and AI integration. The system serves as both a functional collaboration tool and a comprehensive academic project that showcases real-world software engineering practices.

Ease Collaborate

1. Introduction

In today’s fast-paced digital environment, effective collaboration and task management are essential for the success of teams in both academic and professional settings. Many organizations and student groups struggle with organizing tasks, tracking progress, and maintaining accountability when work is distributed across multiple individuals. The absence of a centralized system often leads to miscommunication, missed deadlines, and reduced productivity.

EaseCollaborate is designed to address these challenges by providing a unified platform for managing work in a structured and efficient manner. The system enables users to organize their activities into workspaces, create projects within those workspaces, and manage tasks associated with each project. By integrating collaboration tools with task tracking and analytics, the platform ensures that users can easily monitor progress and stay aligned with team objectives.

The application follows a modern full-stack architecture, combining a responsive frontend with a robust backend API and a scalable database. It also incorporates secure authentication mechanisms and role-based access control to ensure that only authorized users can perform specific actions.

Overall, the system aims to simplify project management while maintaining flexibility, scalability, and usability, making it suitable for both learning environments and real-world applications.

1.1. Background

The rapid growth of collaborative work environments has increased the need for digital tools that can effectively manage tasks, projects, and team interactions. Traditional methods of task tracking, such as spreadsheets or manual lists, are no longer sufficient for handling complex workflows involving multiple users and interdependent tasks.

Over time, various project management tools have been developed to address these challenges. These tools offer features such as task assignment, deadline tracking, and team collaboration. However, many of them are either too complex, costly, or not tailored for learning purposes.

Furthermore, they often abstract away internal implementation details, limiting their usefulness for students who want to understand how such systems are built.

EaseCollaborate is developed as a response to these limitations, combining essential project management features with a transparent and modular architecture. It provides an opportunity to explore modern web development practices while also delivering a practical solution for team collaboration.

1.2. Motivation

The primary motivation behind developing EaseCollaborate was to create a comprehensive and practical project management system that demonstrates real-world full-stack development concepts. Many existing tools provide powerful features, but they often lack simplicity or accessibility for students and small teams.

This project was motivated by the need to:

- build an end-to-end application that integrates frontend, backend, and database systems
- understand and implement authentication and authorization mechanisms
- design scalable and modular system architecture
- explore the integration of artificial intelligence in productivity tools
- create a project suitable for academic evaluation and professional portfolios

Additionally, the idea of incorporating AI-generated summaries was driven by the increasing importance of intelligent systems in improving productivity and decision-making. By combining traditional task management with AI insights, EaseCollaborate aims to provide a more advanced and useful user experience.

1.3. Objectives

The main objectives of EaseCollaborate are as follows:

- To design and develop a centralized platform for project and task management
- To implement secure user authentication and authorization mechanisms
- To enable workspace-based collaboration among multiple users

- To provide structured project and task lifecycle management
- To incorporate role-based access control for secure and organized workflows
- To develop an interactive and user-friendly interface
- To integrate AI-based summarization for productivity insights
- To visualize project progress through dashboards and analytics

These objectives ensure that the system is not only functional but also scalable, secure, and aligned with real-world requirements.

1.4. Scope of the Project

The scope of EaseCollaborate includes the development of a web-based platform that supports user authentication, workspace management, project creation, and task tracking. Users can collaborate within workspaces, assign tasks, update progress, and communicate through comments and activity logs.

The system also includes features such as subtasks, task prioritization, watchers, and dashboard analytics. Additionally, the integration of AI summarization allows users to gain insights into their productivity and workflow patterns.

However, the current scope is limited to a prototype-level implementation suitable for academic and small-scale use. Advanced enterprise-level features such as real-time collaboration, large-scale file storage, complex notification systems, and organization-wide analytics are not included in this version.

The project focuses on demonstrating core software engineering concepts rather than building a fully commercial product.

1.5. Key Features

EaseCollaborate offers a wide range of features designed to support efficient collaboration and task management:

- User registration, login, and authentication
- Email verification and password recovery
- Workspace creation and member management

- Role-based access control at workspace and project levels
- Project creation and lifecycle management
- Task creation, assignment, and status tracking
- Task prioritization, subtasks, and comments
- Personal dashboard with analytics and statistics
- “My Tasks” view for user-specific tracking
- AI-powered summarization for tasks, projects, and workspaces
- User profile management and settings

These features collectively provide a complete environment for managing collaborative work effectively.

1.6. Project Significance

EaseCollaborate is significant as it integrates multiple core areas of software engineering into a single cohesive system. It demonstrates the practical application of frontend development, backend API design, database modeling, authentication, and access control.

The project also highlights the importance of structured collaboration in modern work environments. By providing a centralized platform for managing tasks and projects, it helps improve organization, accountability, and productivity.

Furthermore, the integration of AI-based summarization introduces an innovative element that enhances user experience by providing intelligent insights. This reflects the growing trend of incorporating artificial intelligence into everyday applications.

From an academic perspective, EaseCollaborate serves as a comprehensive project that showcases the ability to design, develop, and document a full-stack application. It is also a valuable addition to a professional portfolio, demonstrating practical skills and real-world problem-solving capabilities.

2. Literature Review

2.1. Overview

Project management systems have become an essential part of modern collaborative environments, enabling teams to organize tasks, monitor progress, and coordinate activities efficiently. With the increasing complexity of software development, academic projects, and organizational workflows, digital tools have evolved to support structured task management, communication, and performance tracking.

Early task management approaches relied on manual methods such as paper-based tracking and spreadsheets. However, these methods lacked scalability, real-time updates, and collaboration features. Modern project management tools have addressed these limitations by introducing centralized platforms that integrate task assignment, deadline tracking, reporting, and team communication.

Recent advancements in this domain have also introduced intelligent features such as automation, predictive analytics, and AI-based summarization. These enhancements aim to reduce manual effort, improve decision-making, and provide deeper insights into productivity patterns. Despite these advancements, there remains a need for systems that balance usability, flexibility, and transparency, especially for educational and small-scale collaborative use.

2.2. Existing Systems

Several project management tools are widely used in industry and academia. These platforms provide a variety of features for task organization, collaboration, and workflow management.

One popular system is **Trello**, which uses a Kanban-style board to organize tasks into lists and cards. It is simple and intuitive, making it suitable for small teams, but it lacks advanced analytics and structured role management.

Another widely used tool is **Jira**, which is designed primarily for software development teams. It offers powerful features such as issue tracking, sprint planning, and workflow customization. However, its complexity can make it difficult for beginners and non-technical users.

Asana provides a balance between usability and functionality, offering task tracking, timelines, and team collaboration features. While it is user-friendly, many advanced features are available only in paid plans.

Notion is a flexible tool that combines note-taking, databases, and task management. Although highly customizable, it requires manual setup and lacks strict workflow enforcement.

Similarly, **Monday.com** offers extensive customization and visual dashboards, but it is often considered expensive and complex for small teams or students.

These systems demonstrate the wide range of approaches available in project management, each with its own strengths and limitations.

2.3. Gaps in Existing Systems

Despite the availability of advanced project management tools, several limitations can be identified, particularly in the context of learning environments and small teams.

Many existing systems are feature-heavy, which can overwhelm users and reduce usability. Beginners often find it difficult to understand complex workflows and configurations, especially in tools designed for enterprise use. Additionally, several platforms operate on subscription-based models, restricting access to advanced features for free users.

Furthermore, while some platforms have started integrating AI features, these are often limited, not customizable, or hidden behind premium plans. There is also limited focus on providing lightweight, meaningful productivity insights tailored to individual users or small teams.

Finally, many tools do not strike a proper balance between simplicity and functionality, either being too basic or excessively complex.

2.4. Need for a New Approach

The limitations of existing systems highlight the need for a project management platform that is both functional and accessible. EaseCollaborate is proposed as a solution that balances simplicity, usability, and architectural clarity while still incorporating essential features required for real-world collaboration.

The new approach focuses on creating a system that is easy to understand and use, particularly for students and small teams, while also demonstrating modern software engineering principles.

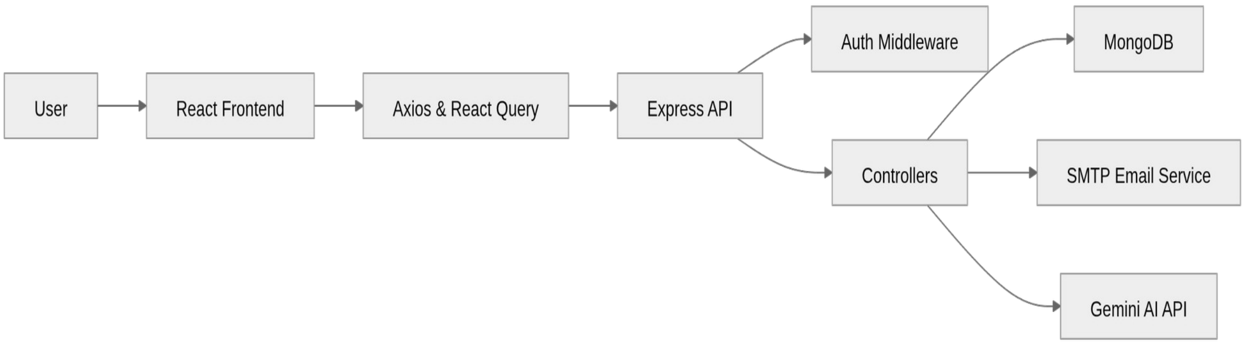
By implementing a modular full-stack architecture, EaseCollaborate provides transparency into how frontend, backend, and database components interact.

In addition, the system integrates AI-based summarization in a practical and focused manner, helping users gain insights into their tasks and productivity without introducing unnecessary complexity. Unlike many existing tools, this feature is designed to be directly useful rather than experimental.

The platform also emphasizes role-based access control, structured workflows, and dashboard analytics, ensuring that users can manage projects efficiently while maintaining clarity and control.

Overall, EaseCollaborate represents a balanced and educationally valuable approach to project management, addressing the gaps identified in existing systems while providing a solid foundation for future enhancements.

3. WorkFlow of the Proposed EaseCollaborate



EaseCollaborate follows a modern **layered client-server architecture**, designed to ensure scalability, modularity, and maintainability. The system is divided into multiple layers, each responsible for a specific set of functionalities, allowing clear separation of concerns.

The architecture primarily consists of a frontend client, a backend server, and a database, along with external service integrations such as email services and AI-based summarization. Communication between these components is achieved through RESTful APIs over HTTP.

3.1. System Components

EaseCollaborate is composed of several interconnected components that work together to provide a complete project management and collaboration platform.

The **frontend component** is built using React and TypeScript. It is responsible for rendering the user interface, handling user interactions, and communicating with the backend through API calls. It includes pages such as authentication, dashboard, workspace view, project view, and task management interfaces.

The **backend component** is developed using Node.js and Express. It handles request processing, implements business logic, enforces authentication and authorization, and manages communication with the database and external services.

The **database component** uses MongoDB to store application data. It maintains collections for users, workspaces, projects, tasks, comments, and activity logs. Mongoose is used to define schemas and manage relationships between these entities.The **authentication component** ensures secure access to the system using JWT-based authentication. It verifies user identity and enforces role-based permissions at both workspace and project levels.

The **email service component** integrates SMTP functionality to support email verification and password reset workflows.The **AI integration component** connects to the Gemini API to generate summaries and insights based on user activity, tasks, and project data.

These components collectively form a modular and scalable system that supports efficient collaboration and task management.

3.2. User Interaction Flow

The user interaction flow in EaseCollaborate describes how a user navigates through the system and performs various actions.

1. A new user registers by providing basic details such as name, email, and password.
2. The system sends an email verification link to the registered email address.
3. After verifying the email, the user logs in and receives a JWT token for authentication.
4. The authenticated user can create a new workspace or join an existing workspace through an invitation.

5. Within a workspace, authorized users can create projects and manage project details.
6. Project managers can create tasks and assign them to team members.
7. Users interact with tasks by updating status, adding comments, managing subtasks, and setting priorities.
8. The dashboard provides an overview of tasks, projects, and productivity statistics.
9. The user can request AI-generated summaries to analyze work progress and performance.
10. The system continuously updates the interface based on user actions and backend responses.

This flow ensures a smooth and structured user experience from registration to advanced task management.

3.3. Backend Communication

EaseCollaborate uses a RESTful API-based communication model between the frontend and backend. The frontend sends HTTP requests to the backend using Axios, while React Query manages data fetching, caching, and synchronization.

Each request includes necessary data and authentication tokens when accessing protected routes. The backend receives these requests and processes them through a series of steps.

Initially, the request passes through middleware, which handles tasks such as parsing JSON data and verifying JWT tokens. Once authenticated, the request is forwarded to the appropriate controller, where business logic is applied.

The controller interacts with the MongoDB database using Mongoose to perform operations such as creating, updating, retrieving, or deleting data. In some cases, the backend also communicates with external services, such as sending emails through SMTP or generating summaries using the Gemini API.

After processing the request, the backend sends a structured JSON response back to the frontend. React Query then updates the UI based on the received data, ensuring that the application remains synchronized with the server state.

This communication model ensures efficient data handling, scalability, and a responsive user experience.

Example Routes :

- `POST /workspaces`

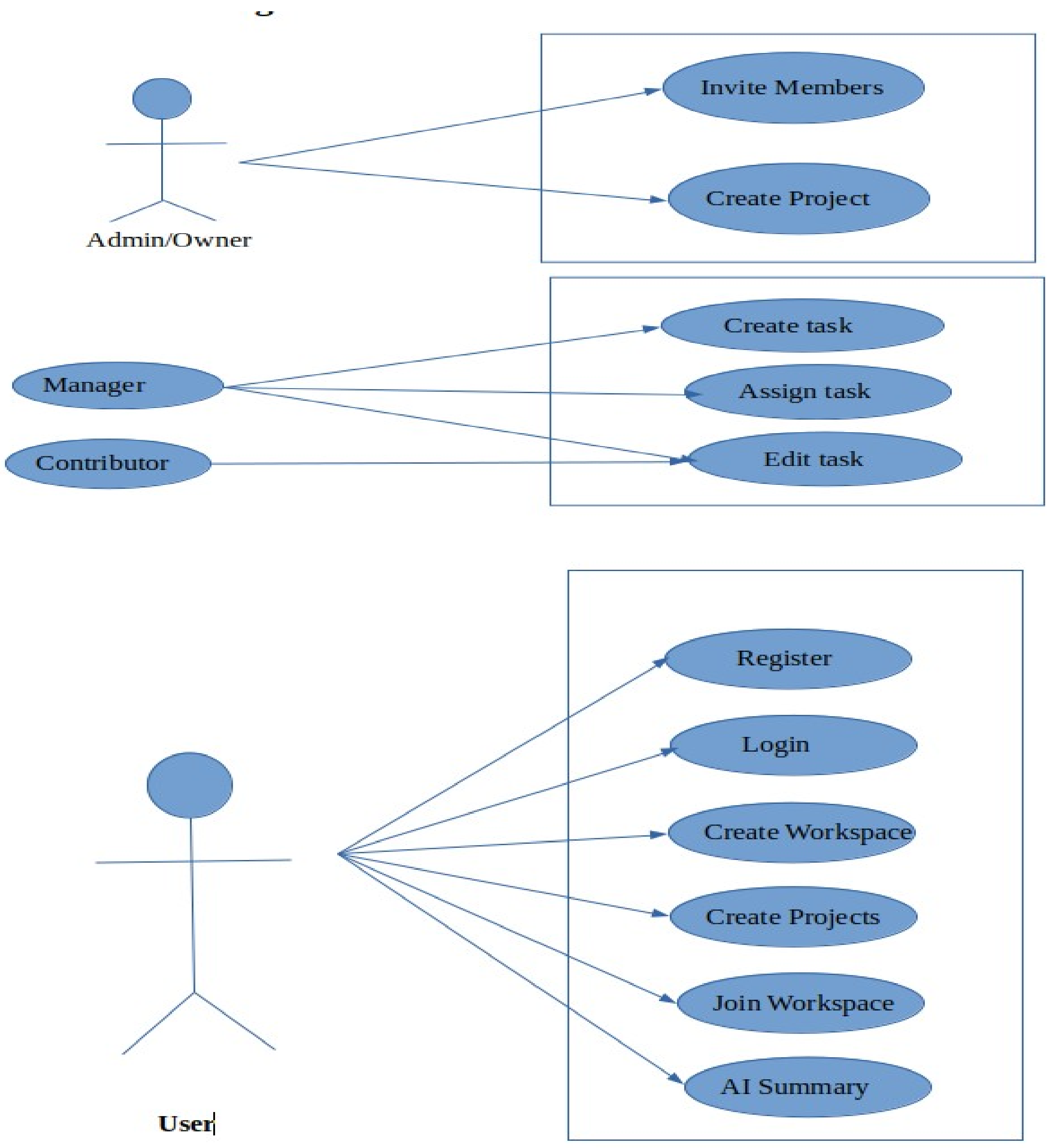
- `GET /workspaces/:workspaceId`

- `GET /projects/:projectId`

- `GET /projects/:projectId/tasks`

- `DELETE /projects/:projectId`

3.4. Use Case Diagram



Use Case Diagram Explanation

The use case diagram represents the interactions between different types of users and the system functionalities in EaseCollaborate.

The primary actor in the system is the user, who can perform actions such as registration, login, viewing profiles, creating or joining workspaces, managing tasks, and generating AI summaries. These actions represent the core functionalities accessible to all authenticated users.

The use case diagram provides a clear visualization of user interactions and system functionalities, helping to understand how different components of the system are utilized in real-world scenarios.

4. Module 1 – User Authentication and Role Management

4.1. Objective

The objective of this module is to ensure secure user authentication, maintain session integrity, and enforce controlled access to system resources using role-based authorization mechanisms.

4.2. Module Description

This module serves as the foundation of the EaseCollaborate system by managing user identity and access control. It handles the complete lifecycle of user authentication, including registration, login, email verification, password recovery, and session handling.

When a user registers, their credentials are processed securely by hashing the password using bcrypt before storing it in the database. An email verification mechanism is implemented to ensure that only valid users gain access to the system. Upon successful verification, users can log in, and a JWT token is generated and sent to the client.

This token is used for subsequent requests and is validated by authentication middleware on the backend. The middleware ensures that only authenticated users can access protected routes, thereby maintaining system security.

In addition to authentication, this module implements role-based access control (RBAC). Users are assigned roles at both workspace and project levels, such as owner, admin, manager, contributor, or viewer. These roles define permissions and restrict actions such as creating projects, assigning tasks, or modifying data.

The module also supports password reset functionality through secure email links, ensuring account recovery without compromising security.

Internal Working

- Passwords are hashed before storage
- JWT tokens are generated on login and validated on each request
- Middleware intercepts requests to verify authentication
- Role checks are applied before executing sensitive operations

4.3. Key Features

- Secure user registration and login
- Email verification system
- JWT-based session management
- Role-based access control (RBAC)

This module ensures that the system is secure, reliable, and capable of handling multiple users with different access levels.

5. Module 2 – Workspace and Collaboration Management

5.1. Objective

The objective of this module is to establish a structured collaborative environment where users can organize work into shared workspaces and manage team interactions effectively.

5.2. Module Description

This module introduces the concept of workspaces, which act as the primary organizational unit within EaseCollaborate. A workspace represents a shared environment where multiple users can collaborate on projects and tasks.

Users can create new workspaces and customize them with attributes such as name, description, and color. Each workspace maintains a list of members, along with their assigned roles. These roles determine the level of access each member has within the workspace.

An invitation system is implemented to allow users to invite others via email. Invitations are managed using secure tokens, which ensure that only authorized users can join a workspace. Once a user accepts an invitation, they are added to the workspace with a predefined role.

The module also provides workspace-level dashboards that display statistics such as the number of projects, tasks, and overall progress. This helps users gain a quick overview of activity within the workspace.

Internal Working

- Workspace data is stored in MongoDB with member-role mappings
- Invitation tokens are generated and validated securely
- Role-based checks are enforced before workspace actions
- Dashboard data is aggregated from related projects and tasks

5.3. Key Features

- Create, update, and delete workspaces
- Invite members via email
- Manage workspace roles (owner, admin, member, viewer)
- View workspace statistics and activity
- Role-based access control within workspaces

This module enables effective collaboration by providing a centralized and controlled environment for team activities.

6. Module 3 – Project and Task Lifecycle Management

6.1. Objective

The objective of this module is to manage the complete lifecycle of projects and tasks, ensuring structured planning, execution, tracking, and completion of work.

6.2. Module Description

This module forms the core of the EaseCollaborate system, handling all operations related to projects and tasks. Projects are created within workspaces and act as containers for organizing tasks.

Each project includes metadata such as title, description, status, start date, due date, and assigned members. Role-based permissions are applied within projects to ensure that only authorized users can create tasks or modify project details.

Tasks are the fundamental units of work within the system. Each task includes attributes such as title, description, status (e.g., To Do, In Progress, Done), priority (Low, Medium, High), due date, and assigned users. Tasks can be further divided into subtasks, enabling detailed tracking of complex activities.

The module supports collaboration through comments, allowing team members to communicate directly within tasks. Users can also watch tasks to receive updates and track changes through activity logs.

The lifecycle of a task includes creation, assignment, progress updates, completion, and archiving. This structured flow ensures that tasks are managed efficiently from start to finish.

Internal Working

- Projects and tasks are linked through database references
- Task updates trigger activity log entries
- Role validation is applied before task modifications
- Subtasks are managed as embedded or related data structures

6.3. Key Features

- Create and manage projects
- Assign roles within projects
- Create, assign, and update tasks
- Set task priorities and deadlines
- Manage subtasks and track completion
- Add comments for collaboration
- Watch tasks and track activity logs
- Archive completed tasks
- “My Tasks” view for personalized tracking

This module ensures efficient task management and provides the tools necessary for tracking progress and collaboration.

7. Module 4 – Dashboard Analytics and AI Summarization

7.1. Objective

The objective of this module is to provide meaningful insights into user activity and improve productivity through data visualization and AI-driven analysis.

7.2. Module Description

This module enhances the functionality of EaseCollaborate by providing analytical and intelligent features. The dashboard displays key performance indicators such as task distribution, project progress, deadlines, and recent activity.

Charts and visual components are used to represent data in an intuitive manner, helping users quickly understand their workload and performance. The dashboard aggregates data from multiple sources, including tasks, projects, and user activity logs.

A key feature of this module is the AI-powered summarization system. Users can request summaries based on different time ranges, such as daily, weekly, or monthly activity. The backend collects relevant data and formats it into prompts, which are then sent to the Gemini API.

The AI processes the data and returns structured summaries that include key highlights, insights, and recommendations. These summaries help users identify trends, prioritize tasks, and improve efficiency.

Internal Working

- Data is aggregated from tasks, projects, and activity logs
- Backend formats data into AI prompts
- Gemini API processes and returns summaries
- Responses are parsed and displayed in the UI

7.3. Key Features

- Interactive dashboard with analytics
- Visualization of task and project data
- Activity tracking and insights
- AI-generated summaries for tasks.
- Time-based analysis (daily, weekly, monthly)

This module adds significant value by transforming raw data into actionable insights, enabling users to make informed decisions and improve productivity.

8. Implementation

EaseCollaborate is implemented as a modular full-stack web application that separates frontend and backend responsibilities while ensuring seamless communication between components. The system follows modern development practices, including component-based UI design, RESTful API architecture, and schema-driven database modeling.

The implementation is divided into two major parts: the frontend and the backend. The frontend is responsible for user interaction and presentation, while the backend handles business logic, authentication, data processing, and integration with external services. MongoDB is used as the database to persist application data.

The application ensures scalability and maintainability by organizing code into reusable components, structured routes, and clearly defined layers. This modular design allows easy debugging, testing, and future enhancements.

8.1. Frontend Implementation

The frontend of EaseCollaborate is developed using React with TypeScript and built using Vite for fast development and optimized builds. The user interface is designed to be responsive, interactive, and user-friendly.

Routing is handled using React Router, enabling navigation between different pages such as login, signup, dashboard, workspaces, projects, and tasks. The application follows a component-based architecture, where reusable UI components such as buttons, forms, modals, and cards are used across multiple pages.

State management for server data is handled using React Query, which simplifies data fetching, caching, and synchronization with the backend. Axios is used to send HTTP requests to the backend API, and an interceptor is implemented to automatically attach JWT tokens to authenticated requests.

Authentication state is managed using a context-based approach, ensuring that protected routes are accessible only to logged-in users. The frontend also includes form validation, error handling, and loading states to improve user experience.

Styling is implemented using Tailwind CSS and shadcn/ui components, providing a clean and consistent design. Dashboard components display analytics such as task distribution, project progress, and activity trends.

The AI summarization feature is integrated into the frontend through a dedicated interface, allowing users to request summaries and view generated insights in a structured format.

8.2. Backend Implementation

The backend of EaseCollaborate is built using Node.js and Express, providing a RESTful API for handling client requests. The backend is responsible for implementing business logic, managing authentication, enforcing permissions, and interacting with the database.

The server is initialized with middleware for JSON parsing, CORS handling, and request logging. Routes are organized into separate modules such as authentication, users, workspaces, projects, tasks, and AI summarization.

Authentication is implemented using JWT tokens. When a user logs in, a token is generated and sent to the client. This token is then used to access protected routes. Middleware is used to verify tokens and enforce role-based access control.

Data validation is handled using Zod, ensuring that all incoming requests meet the required schema before processing. This improves reliability and prevents invalid data from entering the system.

MongoDB is used as the database, with Mongoose providing schema definitions and query handling. Collections are defined for users, workspaces, projects, tasks, comments, and activity logs. Relationships between these entities are maintained through references.

The backend also integrates external services. Nodemailer is used to send email verification and password reset links, while the Gemini API is used to generate AI-based summaries. The system includes a basic in-memory rate limiter to control the number of AI requests per user.

Controllers are used to encapsulate business logic, making the codebase modular and maintainable. Each API endpoint processes requests, performs necessary validations, interacts with the database, and returns structured JSON responses.

Overall, the backend implementation ensures security, scalability, and efficient data handling.

8.3. Integration of Frontend and Backend

The frontend and backend of EaseCollaborate are integrated through RESTful API communication. The frontend sends HTTP requests using Axios to specific API endpoints exposed by the backend.

Each request includes necessary data and, in the case of protected routes, a JWT token for authentication. The backend processes these requests, interacts with the database, and returns responses in JSON format.

React Query on the frontend manages these responses by updating the UI automatically based on the latest data. This ensures consistency between the client and server states.

Error handling is implemented on both sides to manage issues such as invalid input, authentication failures, and server errors. This integration ensures a smooth and responsive user experience.

The separation of frontend and backend allows independent development and deployment, making the system flexible and scalable.

Key Implementation Highlights :

- Modular code structure for both frontend and backend
- Secure JWT-based authentication system
- Role-based access control enforcement
- Efficient server-state management using React Query
- Schema validation using Zod
- Clean API design using REST principles
- Integration with external services (SMTP and AI API)
- Responsive UI design with reusable components
- Scalable MongoDB database design

These implementation choices ensure that EaseCollaborate is robust, maintainable, and aligned with modern web development standards.

9. Code Implementaion and Output ScreenShots

The implementation of EaseCollaborate involves both backend and frontend components, supported by database setup and integration with external services. Each part of the system is designed to handle specific responsibilities, ensuring smooth communication between the user interface, application logic, and data storage. The code and output screenshots are as follows:

Code Images:

Schema :

```
export const signUpSchema = z
  .object({
    name: z.string().min(3, "Name must be at least 3 characters"),
    email: z.email("Invalid email address"),
    password: z.string().min(8, "Password must be 8 characters"),
    confirmPassword: z.string().min(8, "Password must be 8 characters"),
  })
  .refine((data) => data.password === data.confirmPassword, {
    path: ["confirmPassword"],
    message: "Passwords do not match",
  });

export const resetPasswordSchema = z
  .object({
    newPassword: z.string().min(8, "Password must be 8 characters"),
    confirmPassword: z.string().min(8, "Password must be 8 characters"),
  })
  .refine((data) => data.newPassword === data.confirmPassword, {
    path: ["confirmPassword"],
    message: "Passwords do not match",
  });
```

Routes:

```
      {
        element: <AuthLayout />,
        children: [
          { path: "login", element: <Login /> },
          { path: "signup", element: <Signup /> },
          { path: "verify-email", element: <VerifyEmail /> },
          { path: "forgot-password", element: <ForgotPassword /> },
          { path: "reset-password", element: <ResetPassword /> },
        ],
      },

      //  Dashboard Layout
      {
        element: <DashboardLayout />,
        children: [
          { path: "dashboard", element: <Dashboard /> },
          { path: "workspaces", element: <Workspaces /> },
          { path: "workspaces/:workspaceId", element: <WorkspaceDetails /> },
          {
            path: "workspaces/:workspaceId/projects/:projectId",
            element: <ProjectDetails />,
          },
          {
            path: "workspaces/:workspaceId/projects/:projectId/edit",
            element: <EditProject />,
          },
          {
            path: "workspaces/:workspaceId/projects/:projectId/tasks/:taskId",
            element: <TaskDetails />,
```

Functions For Updating Details :

```
export const UseCreateProject = () => {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: async (data: {
      projectData: CreateProjectFormData;
      workspaceId: string;
    }) =>
      postData(
        `/projects/${data.workspaceId}/create-project`,
        data.projectData
      ),
    onSuccess: (data: any) => {
      queryClient.invalidateQueries({
        queryKey: ["workspace", data.workspace],
      });
    },
  });
};

export const UseProjectQuery = (projectId: string) => {
  return useQuery({
    queryKey: ["project", projectId],
    queryFn: () => fetchData(`/projects/${projectId}/tasks`),
  });
};

export const useProjectDetailsQuery = (projectId: string) => {
  return useQuery({
    queryKey: ["project-details", projectId],
    queryFn: () => fetchData(`/projects/${projectId}`),
  });
};
```

Controllers :

```
const getWorkspaces = async (req, res) => {
  try {
    const workspaces = await Workspace.find({
      "members.user": req.user._id,
    }).sort({ createdAt: -1 });

    res.status(200).json(workspaces);
  } catch (error) {
    console.log(error);
    res.status(500).json({
      message: "Internal server error",
    });
  }
};

const getWorkspaceDetails = async (req, res) => {
  try {
    const { workspaceId } = req.params;

    const workspace = await Workspace.findById({
      _id: workspaceId,
    }).populate("members.user", "name email profilePicture");

    if (!workspace) {
      return res.status(404).json({
        message: "Workspace not found",
      });
    }

    res.status(200).json(workspace);
  } catch (error) {}
};
```

Backend Routes :

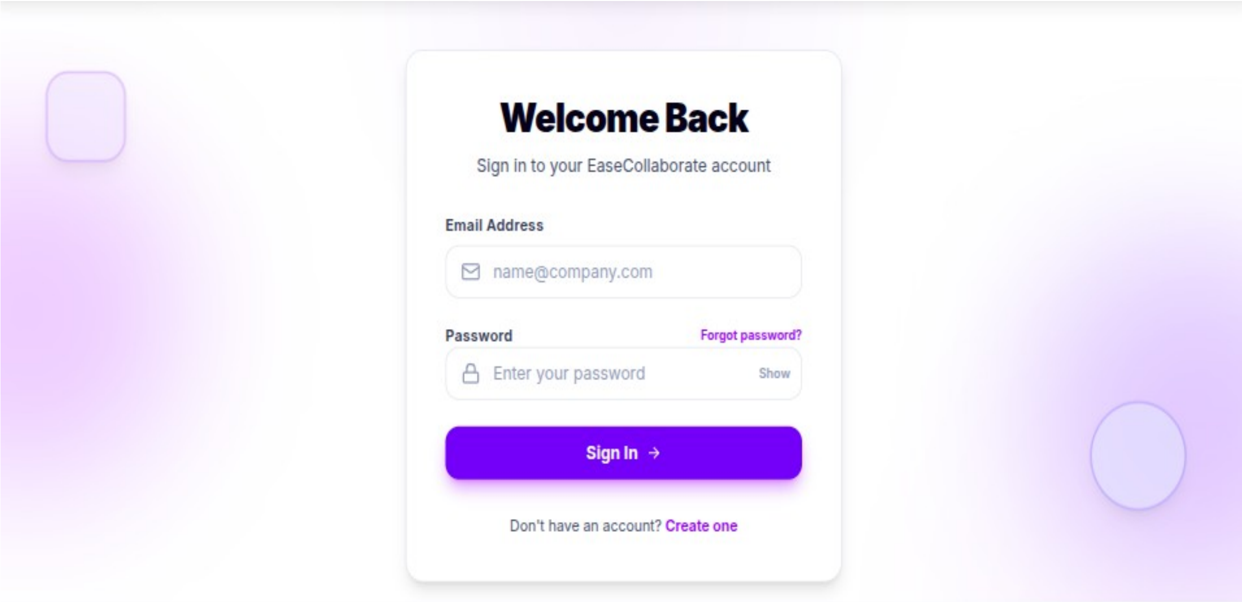
```
router.post(
  "/accept-invite-token",
  authMiddleware,
  validateRequest({ body: tokenSchema }),
  acceptInviteByToken
);

router.post(
  "/:workspaceId/invite-member",
  authMiddleware,
  validateRequest({
    params: z.object({ workspaceId: z.string() }),
    body: inviteMemberSchema,
  }),
  inviteUserToWorkspace
);

router.post(
  "/:workspaceId/accept-generate-invite",
  authMiddleware,
  validateRequest({ params: z.object({ workspaceId: z.string() }) }),
  acceptGenerateInvite
);

router.get("/", authMiddleware, getWorkspaces);

router.get("/:workspaceId", authMiddleware, getWorkspaceDetails);
router.get("/:workspaceId/projects", authMiddleware, getWorkspaceProjects);
router.get("/:workspaceId/stats", authMiddleware, getWorkspaceStats);
```

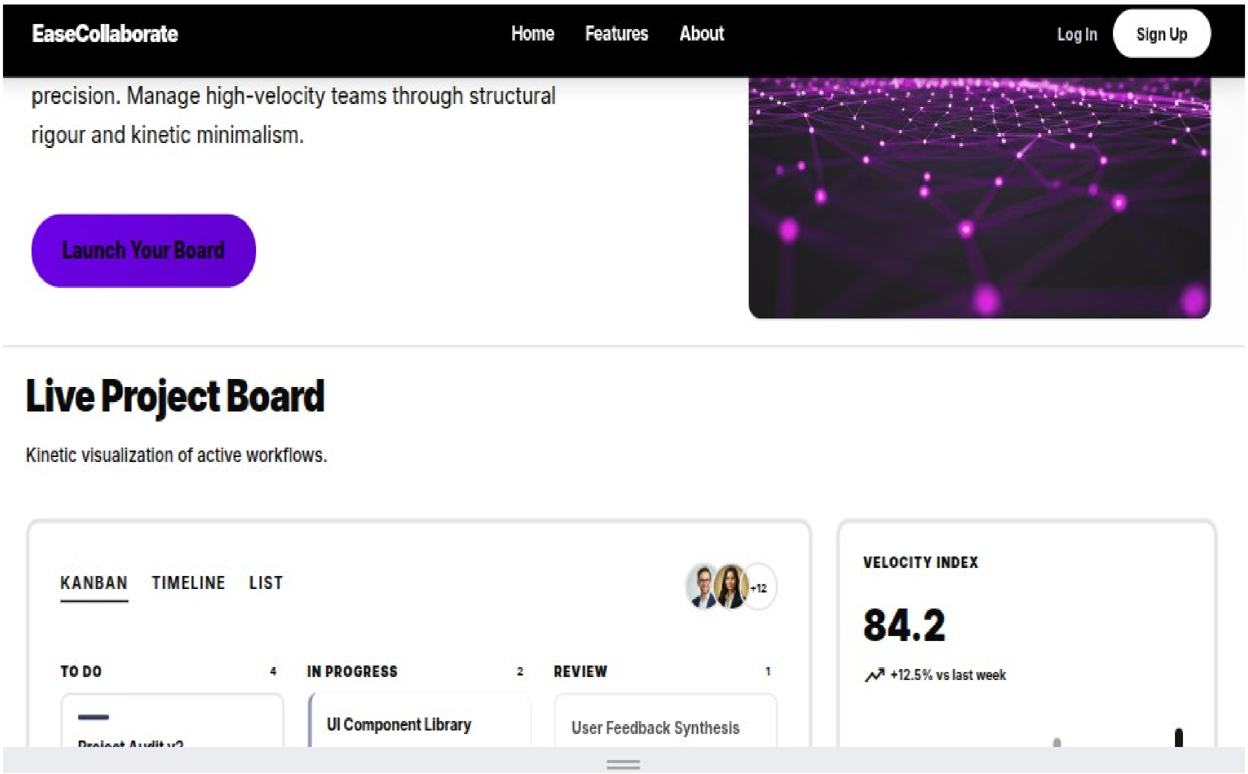
controller:

```
const verifyEmail = async (req, res) => {
  try {
    const { token } = req.body;

    const payload = jwt.verify(token, process.env.JWT_SECRET);

    if (!payload) {
      return res.status(401).json({ message: "Unauthorized" });
    }

    const { userId, purpose } = payload;

    if (purpose !== "email-verification") {
      return res.status(401).json({ message: "Unauthorized" });
    }

    const verification = await Verification.findOne({
      userId,
      token,
    });

    if (!verification) {
      return res.status(401).json({ message: "Unauthorized" });
    }
```

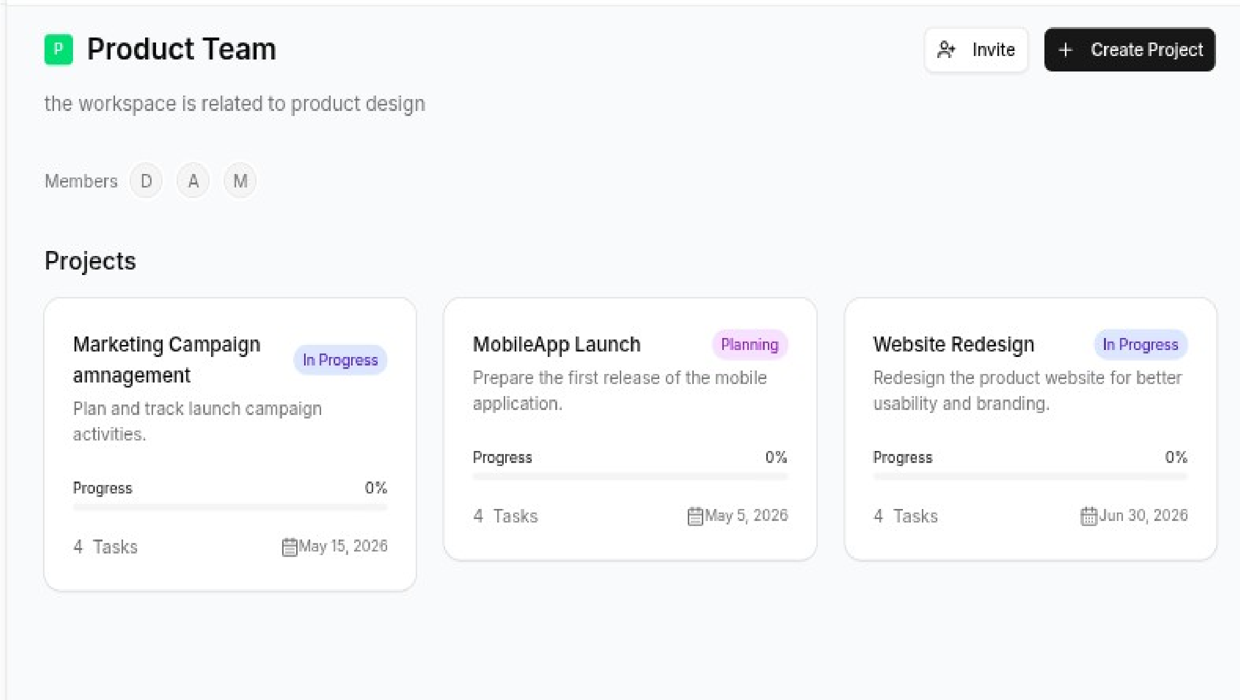
OUTPUT :

Login an Marketing Page :

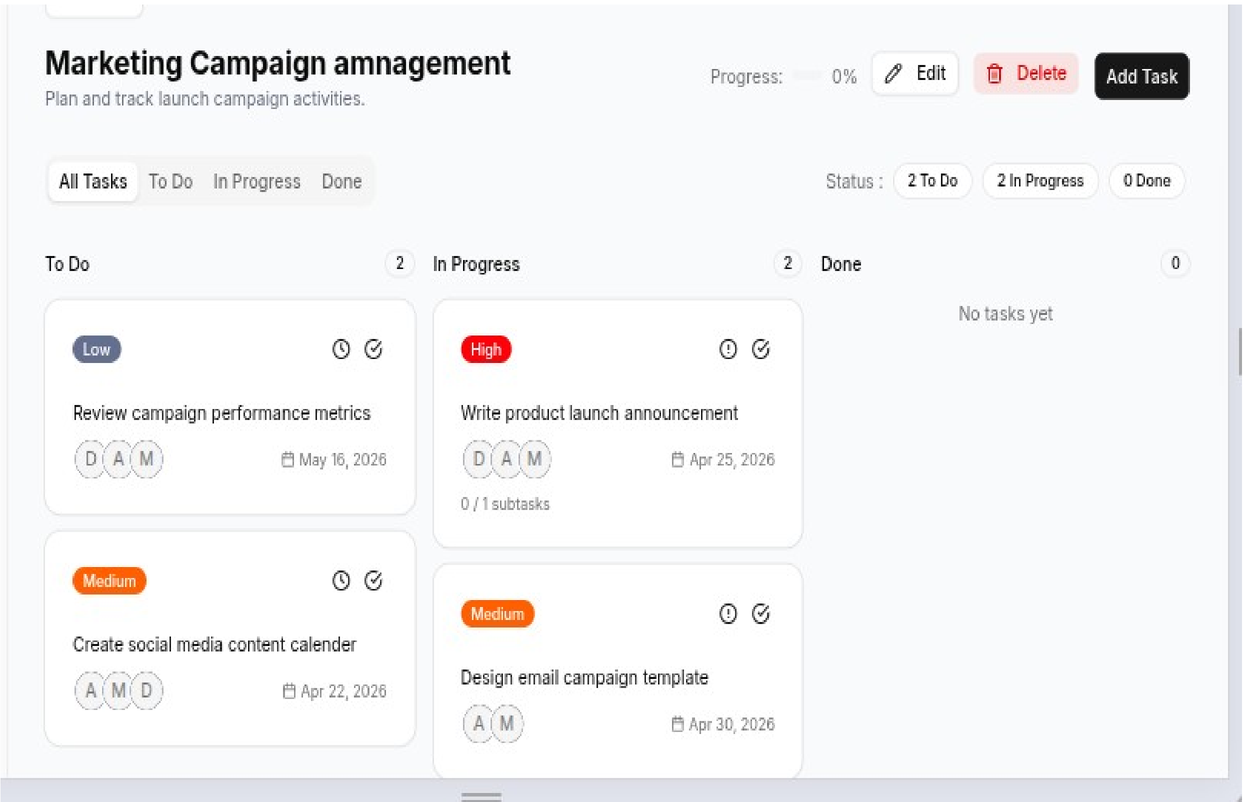




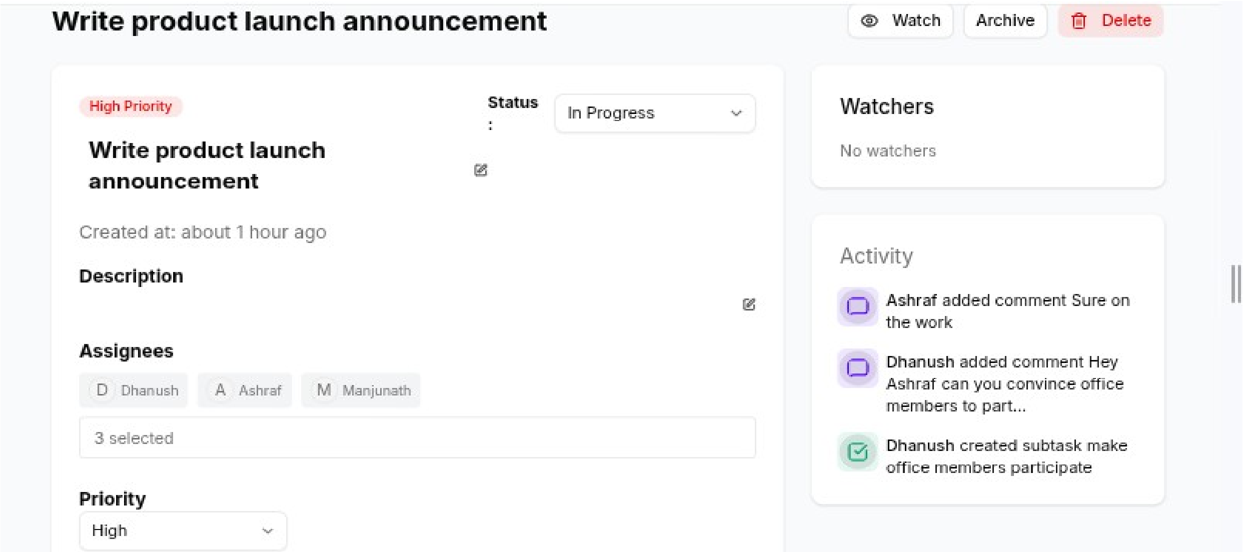
Projects Page :



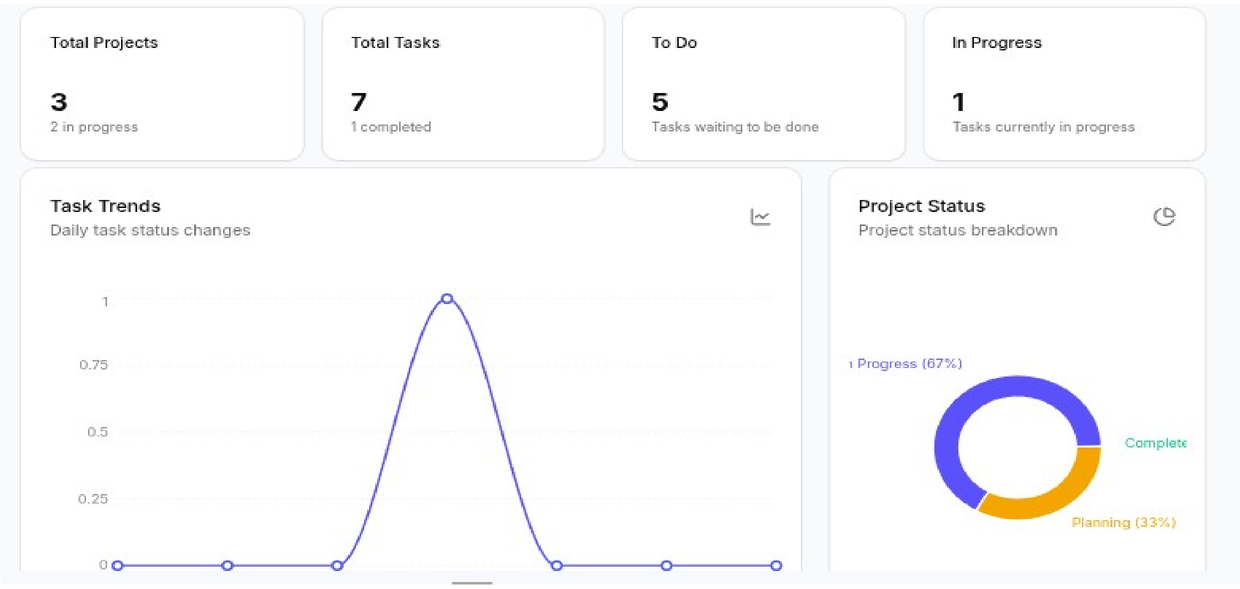
Task Page :

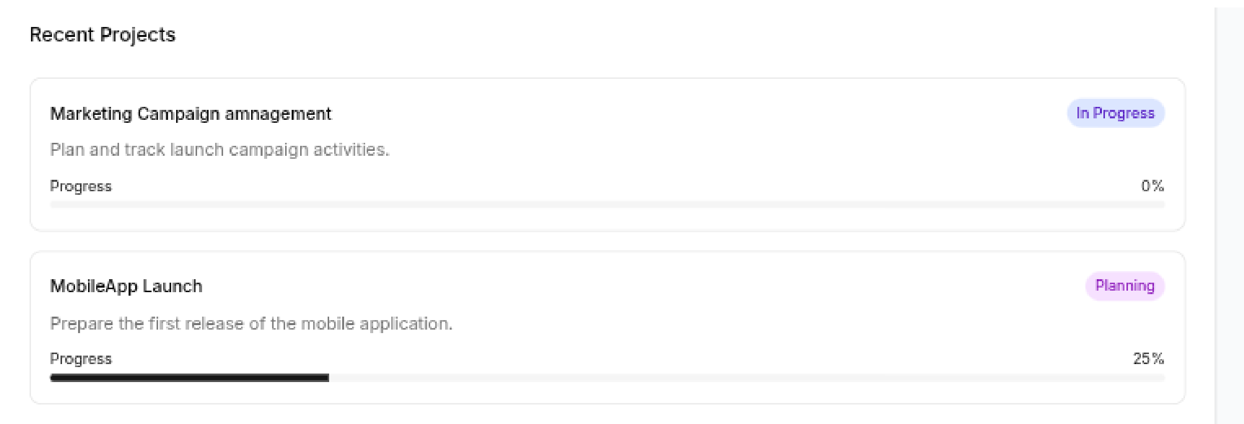


Task Details Page:

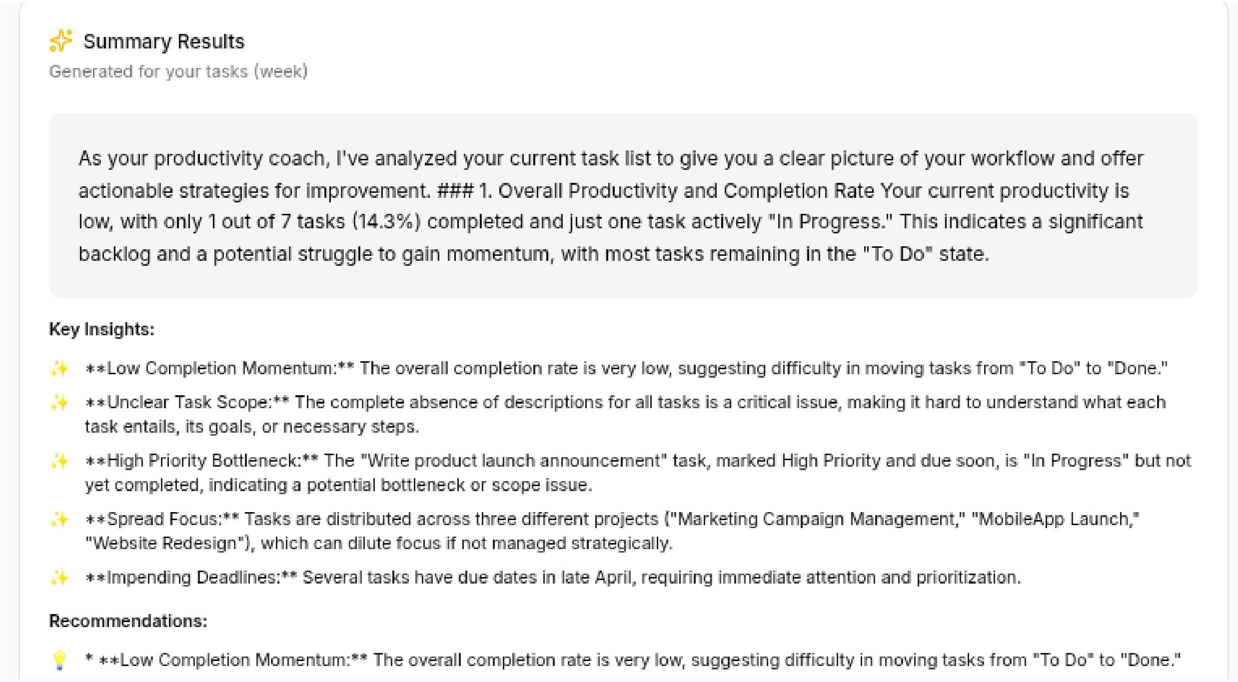


Dashboard Pages:





AI Summarization Page :



10. Future Enhancements

Although EaseCollaborate provides a comprehensive set of features for project management and collaboration, there are several areas where the system can be further enhanced to improve functionality, scalability, and user experience.

One major enhancement would be the implementation of **real-time collaboration** using technologies such as WebSockets. This would allow users to see updates instantly, such as task changes, comments, and activity logs, without needing to refresh the page.

Another important improvement is the addition of a **notification system**. Users could receive real-time alerts for task assignments, comments, deadlines, and workspace invitations, improving communication and responsiveness within teams.

The system can also be extended to support **file and attachment uploads**, allowing users to attach documents, images, or other resources to tasks and projects. Integration with cloud storage services could further enhance this capability.

To improve scalability, the AI summarization module can be enhanced with **persistent rate limiting and usage tracking**, ensuring consistent performance even under high usage. More advanced AI features such as task prioritization suggestions and predictive analytics can also be introduced.

Additional enhancements include:

- Mobile application development for better accessibility
- Advanced analytics and reporting dashboards
- Role-based administrative controls at a higher organizational level
- Integration with third-party tools such as calendars and communication platforms
- Improved UI/UX design for accessibility and responsiveness

These enhancements would transform EaseCollaborate from a prototype-level system into a more robust and enterprise-ready platform.

11. Conclusion

EaseCollaborate successfully achieves its goal of providing a structured, secure, and user-friendly project management and collaboration platform. The system integrates essential features such as user authentication, workspace management, project organization, and task tracking into a unified application.

The implementation demonstrates the effective use of modern full-stack technologies, including React for the frontend, Express for the backend, and MongoDB for data management. The system also incorporates role-based access control to ensure secure and organized workflows.

One of the key strengths of the project is the integration of AI-based summarization, which enhances user productivity by providing meaningful insights into tasks, projects, and overall activity. The dashboard analytics further support users in tracking progress and understanding workload distribution.

In conclusion, EaseCollaborate serves as both a functional collaboration tool and a comprehensive demonstration of full-stack development skills. The system provides a solid foundation for future improvements and can be extended to meet more advanced requirements in real-world applications.

12. References

The development of EaseCollaborate is based on various technologies, frameworks, and documentation resources. The following references were used during the design and implementation of the system:

- React Documentation – https://react.dev
- Vite Documentation – https://vitejs.dev
- React Router Documentation – https://reactrouter.com
- TanStack React Query Documentation – https://tanstack.com/query
- Tailwind CSS Documentation – https://tailwindcss.com
- Node.js Documentation – https://nodejs.org
- Express.js Documentation – https://expressjs.com
- MongoDB Documentation – https://www.mongodb.com/docs
- Mongoose Documentation – https://mongoosejs.com
- Zod Documentation – https://zod.dev
- Nodemailer Documentation – https://nodemailer.com
- Google Gemini API Documentation – https://ai.google.dev
- JWT (JSON Web Token) Documentation – https://jwt.io

These resources provided guidance for implementing various features such as frontend development, backend API design, database integration, authentication, and AI-based summarization.